

Computational Fluid Dynamics for Aerospace Applications

Lecture 7

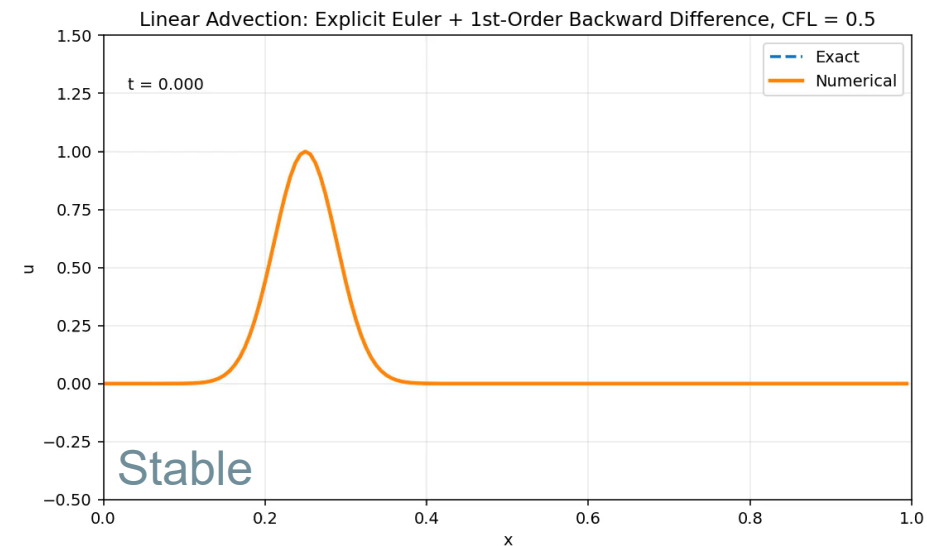
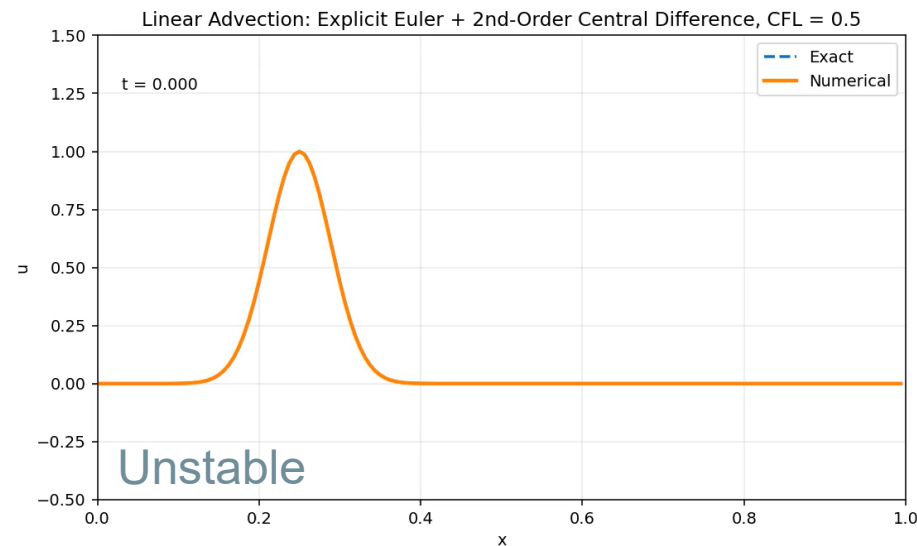
Stability and Boundary Conditions

Stability and Boundary Conditions

- Stability
- Dissipation
- Dispersion
- Boundary Conditions

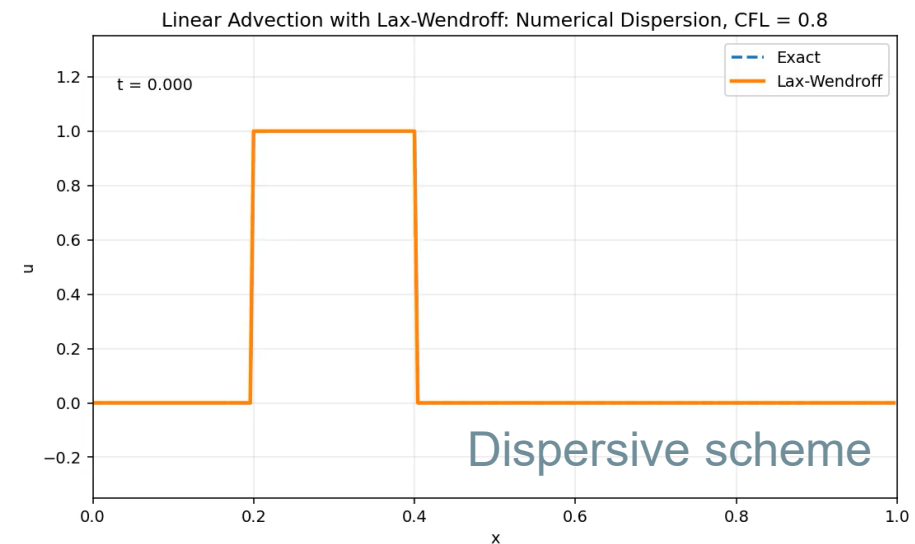
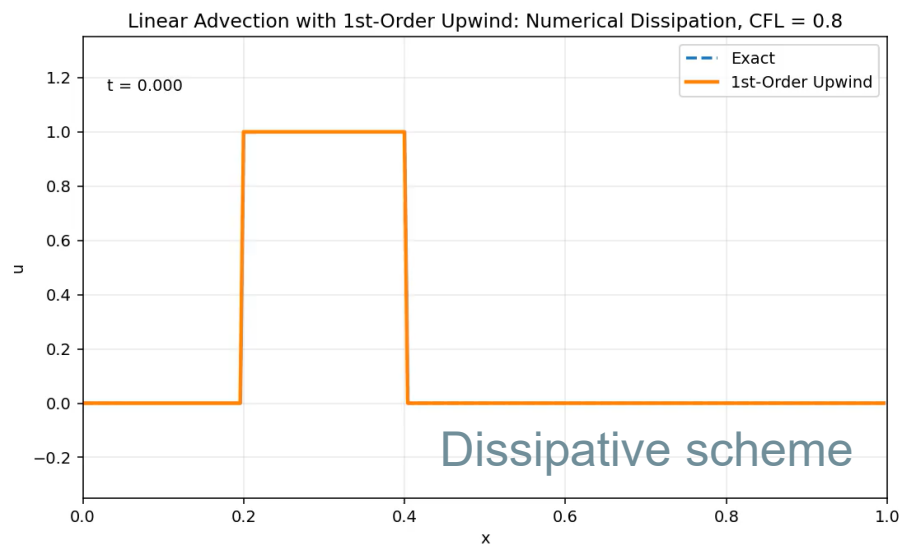
Stability

- Last lecture we covered stability, and stated that a stable scheme is one where small errors introduced into the solution do not grow without bound as the computation advances



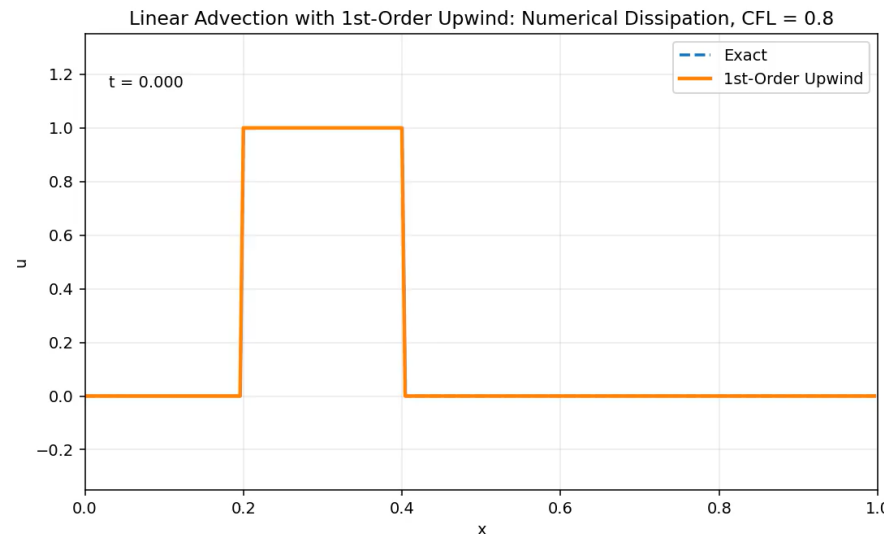
Stability

- Stability is a necessary condition for a “good” simulation
- However, even a stable scheme can still distort the solution and lead to a “bad” simulation
 - Distortion of shape and amplitude
 - Distortion of propagation speed

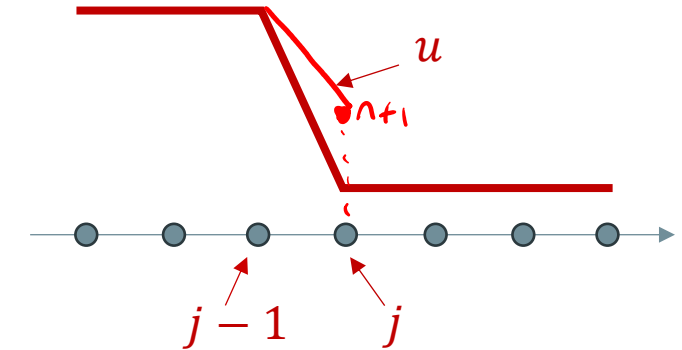


Dissipation

- Numerical dissipation artificially reduces gradients and amplitudes (smears the solution)
 - Note that this is the same as the physical concept of diffusion
 - We often refer to dissipation as “numerical diffusion” as a result
 - In terms of error, numerical dissipation is like overestimating diffusion

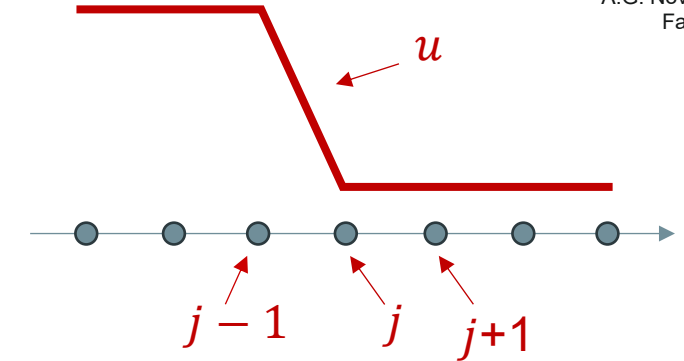


Dissipation



- Why does numerical dissipation happen?
- Consider linear advection (again...) $\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0$
- Let's discretize with EE FO BD $u_j^{n+1} = u_j^n - \frac{a\Delta t}{\Delta x} (u_j^n - u_{j-1}^n)$
 - This equation essentially just replaces the quantity at j with an averaged version of the same quantity at j
 - For example, consider $\frac{a\Delta t}{\Delta x} = 0.5$, then $u_j^{n+1} = \frac{1}{2}u_j^n + \frac{1}{2}u_{j-1}^n$
 - If u_{j-1} is bigger than u_j , this scheme will always work to increase u_j but decrease u_{j-1} until they are equal
- Dissipation fundamentally unavoidable with such a scheme

Dissipation



- Consider linear advection (again...) $\frac{\partial u}{\partial t} + a \frac{\partial u}{\partial x} = 0$
- A non-dissipative example: Leapfrog + SO central difference

Leapfrog approximation
to $\partial u / \partial t$

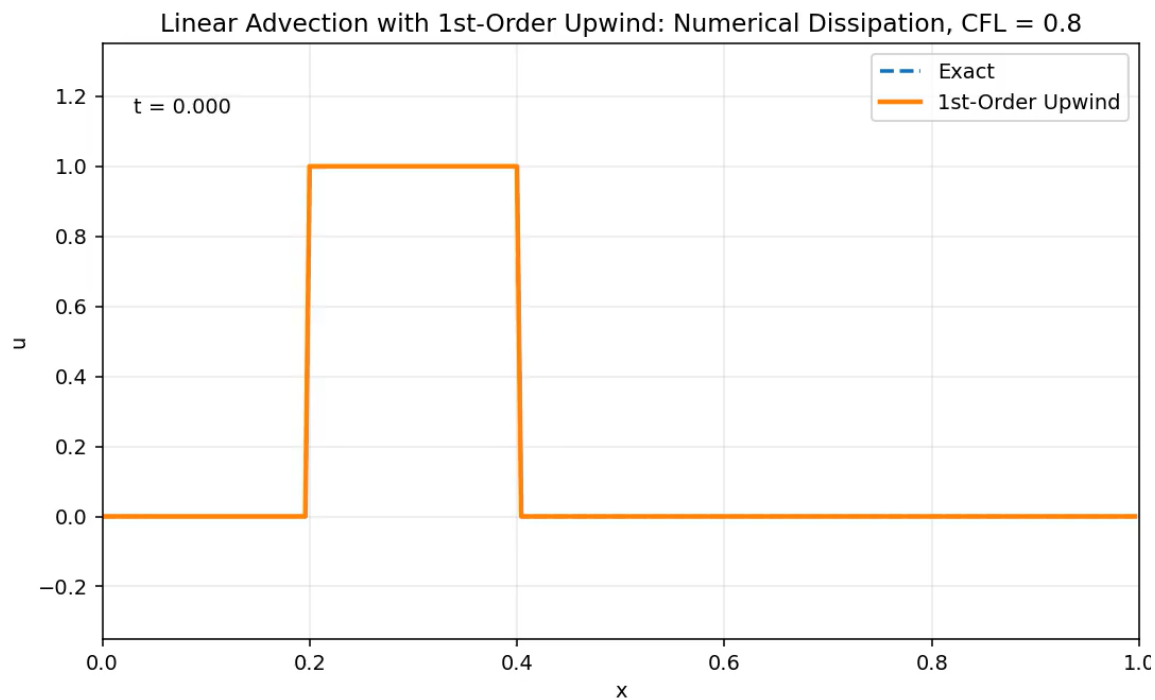
$$\frac{u_j^{n+1} - u_j^{n-1}}{2\Delta t} + a \frac{u_{j+1}^n - u_{j-1}^n}{2\Delta x} = 0$$

$$u_j^{n+1} = u_j^{n-1} - CFL \times (u_{j+1}^n - u_{j-1}^n)$$

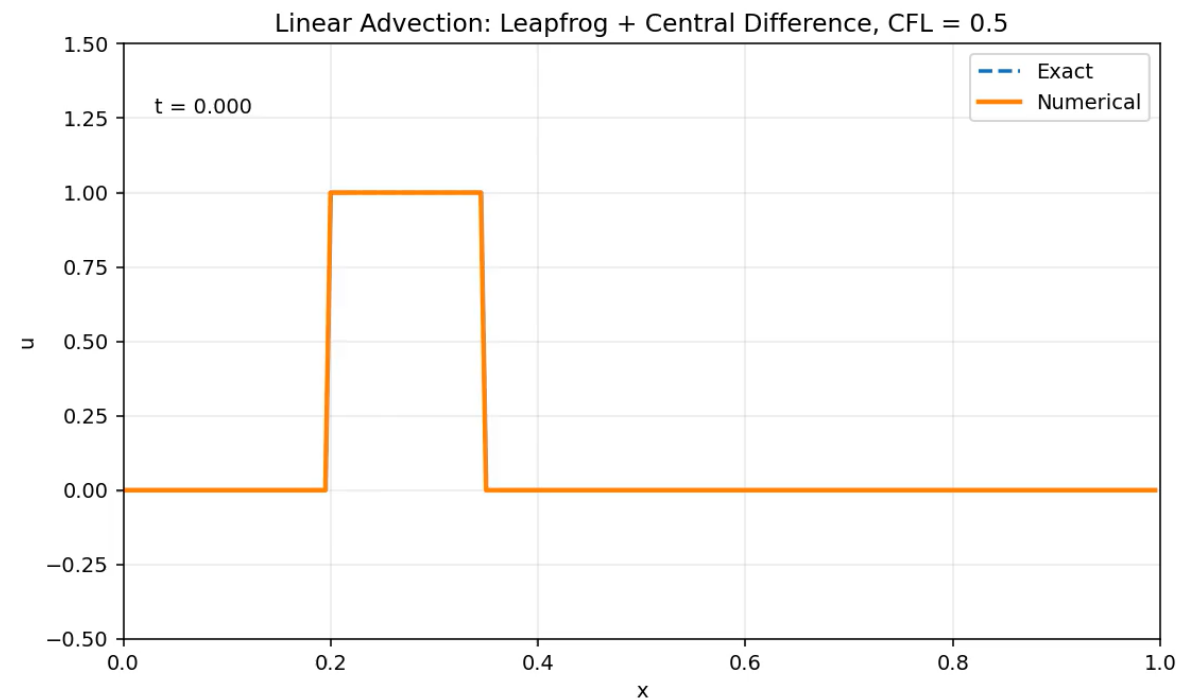
- The value of u at point j at the next timestep is not itself averaged with a neighboring value
- This temporal+spatial scheme combination is non-dissipative

Dissipation

EE FO Upwind
(Dissipative)



Leapfrog SO Central
(Not Dissipative! But clearly struggling...)



Dispersion

- What is going on in the video on the previous slide?
 - We found a non-dissipative scheme, but it's unfortunately dispersive
- Numerical Dispersion causes different wavelengths of the solution to travel at incorrect speeds
- Recall that Fourier Series allows us to express any function (e.g., our solution u) as the sum of infinite waves

$$u(x) = \sum_{n=-\infty}^{\infty} c_n e^{i \frac{n\pi x}{L}}$$

$$e^{i\theta} = \cos\theta + i\sin\theta$$

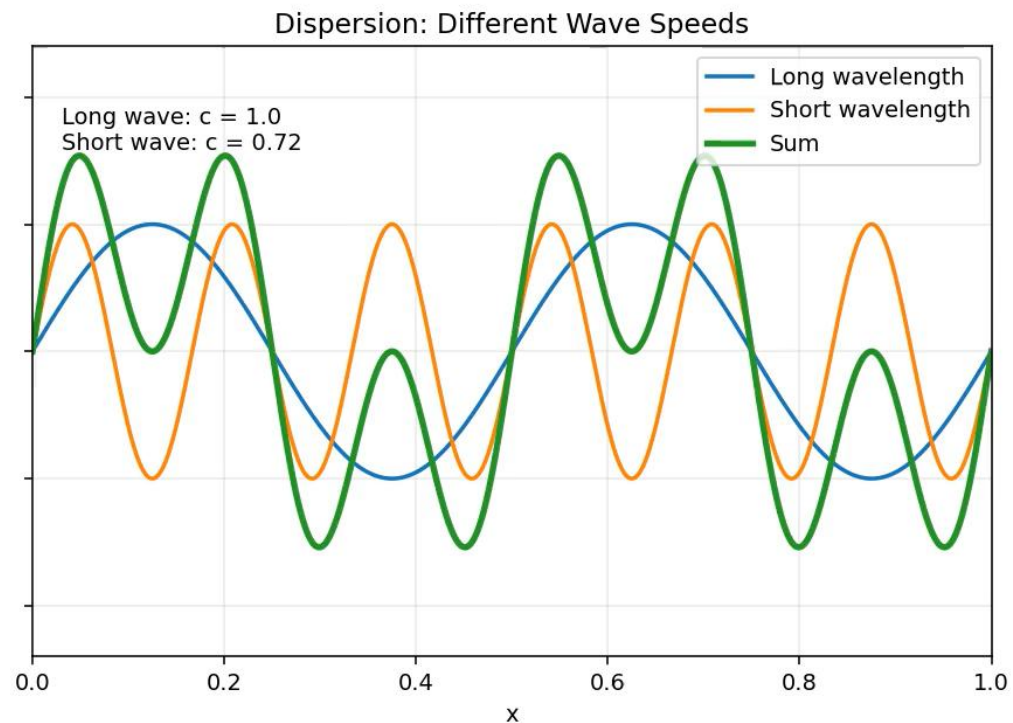
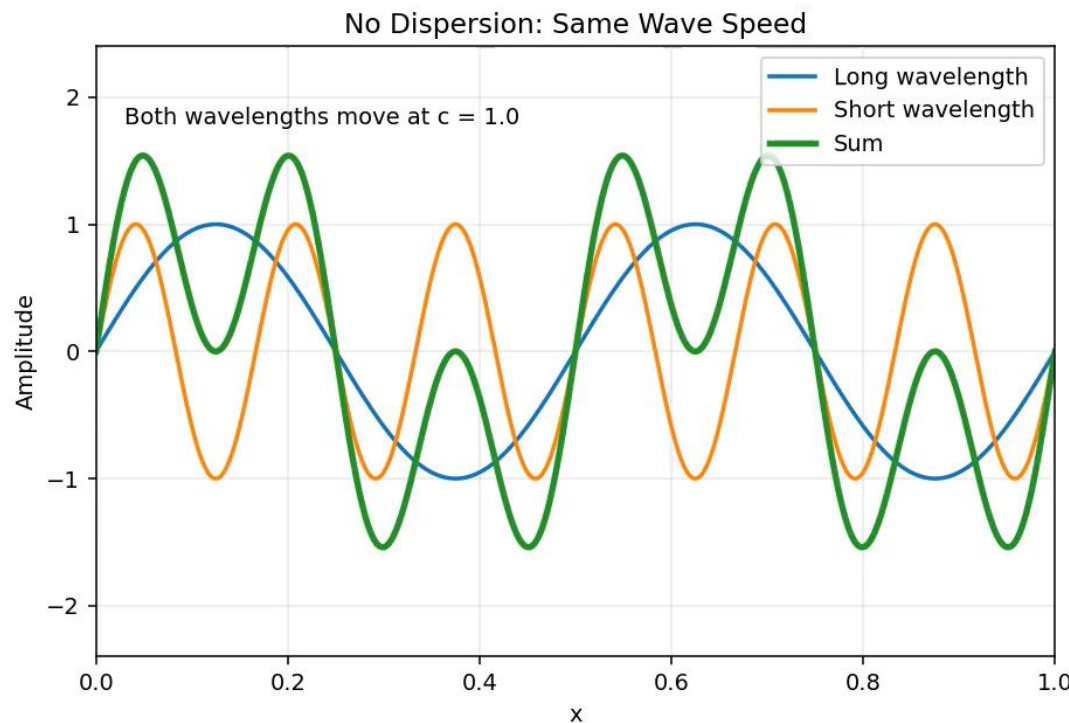
- What happens to the solution if these waves don't all advect at the same exact speed?

Dispersion

$$u(x) = \sum_{n=-\infty}^{\infty} c_n e^{i \frac{n\pi x}{L}}$$

- Different advection speeds for different wavelengths leads to an inconsistent (and incorrect) solution

Two-Wave Superposition: $t = 0.00$



Dispersion

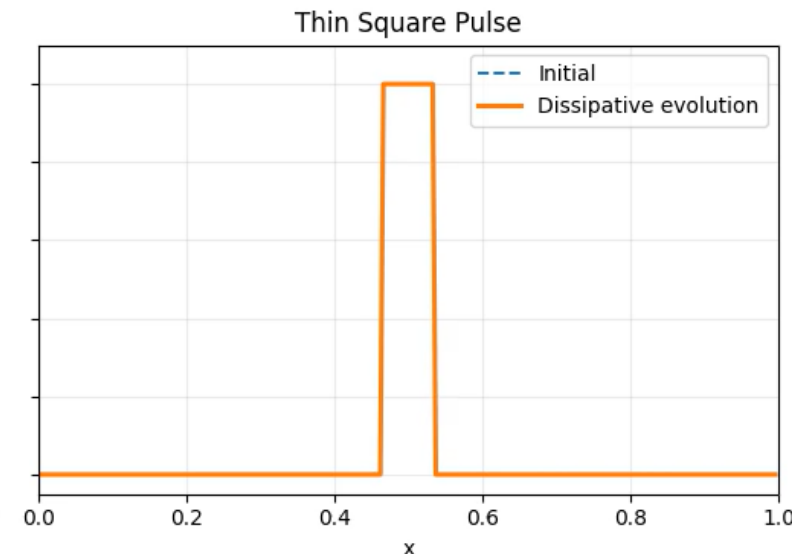
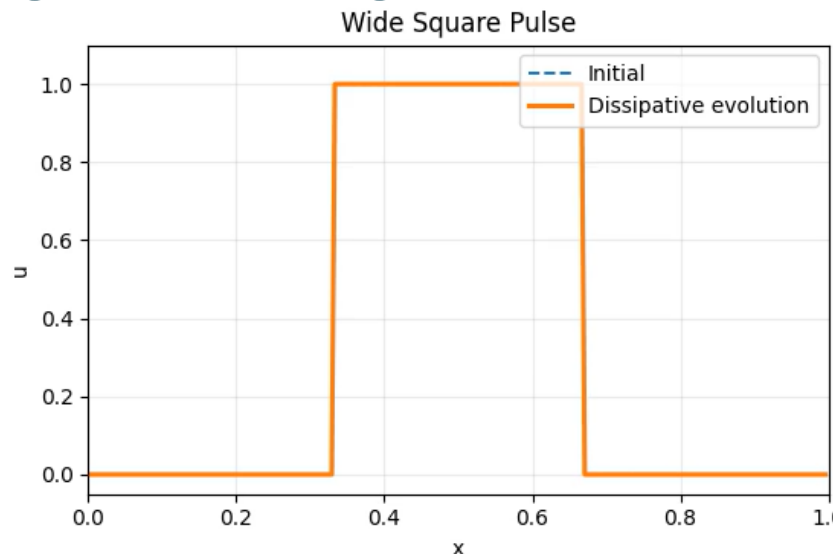
$$c = \frac{a}{\Delta x}$$

k = Wavenumber

- We won't go into the math, but briefly discuss results:
- The true PDE says each wavenumber travels at speed $a \neq a(k)$
- A dispersive numerical scheme says each wavenumber travels at $a = a(k)$
 - For example, SO CD gives $a(k) = a_0 \frac{\sin(k\Delta x)}{k\Delta x}$
 - Note in the above, when $k\Delta x \ll 1 \rightarrow c(k) \approx c_0$
 - This means long wavelengths (small k) travel near the correct speed while short wavelengths (large k) travel at incorrect speeds

Dispersion

- How can we minimize the amount of dispersion? Dissipation!
- We noted that shorter wavelengths suffered the most dispersion
- Dissipation has a stronger effect on shorter wavelengths
 - Use it to damp the problematic wavelengths, without affecting the longer wavelengths as much



Dispersion

- There is always going to be trade-off in your choice of numerics
- Dissipation can damp dispersion, but has its own problems
- Would you rather calculate a wave moving:
 - The correct speed (low dispersion, high dissipation)?
 - Or with the right energy (low dissipation, high dispersion)?
- Better numerical schemes and algorithms can give us lower dispersion AND dissipation
 - The trade-off is programming complexity and simulation cost
- Generally: Low-order/upwind schemes are dissipative, centered schemes are dispersive

Dispersion

- What about second derivatives?
 - These terms are fundamentally dissipative (physical diffusion)
- Using an upwind scheme would add more dissipation
- A centered scheme would add dispersion, but this will already be weakened by the presence of diffusion
- The natural choice for second derivatives is to use a centered approximation

- **Never, ever, ever, ever, ever** use a non-centered scheme for a second derivative!!!!

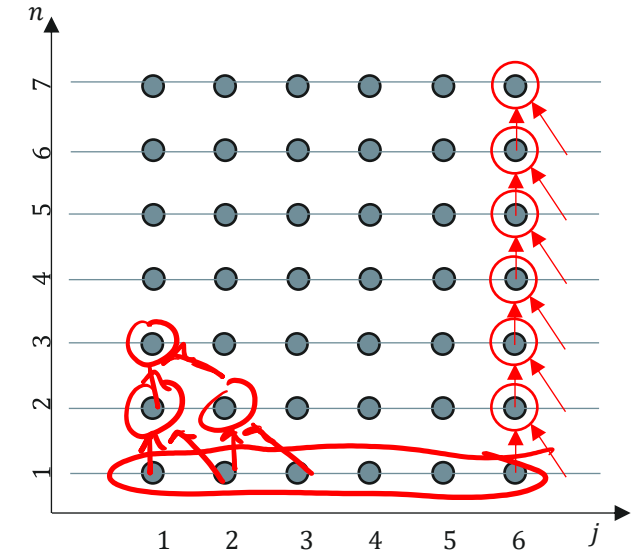
My Ph.D. advisor
when he taught CFD

Stability

- We can now assess whether a discretization scheme is stable
 - We've learned that stability is a strict requirement
- We've learned to identify dissipation
 - Acts like diffusion, weakens the amplitude or “energy” of solution
- We've learned to identify dispersion
 - Modifies the speed of waves in our simulations
- Note that a stable discretization scheme does not guarantee anything about dissipation or dispersion
- Can we do CFD yet?

Boundary Conditions

- We are missing one major piece of the puzzle
- Recall explicit euler forward difference
- $u_j^{n+1} = u_j^n - \frac{a\Delta t}{\Delta x} (u_{j+1}^n - u_j^n)$
- Can we ever solve for the rightmost point?
- We will have a similar issue with any scheme
 - A centered scheme will have the same problem on the left AND right
 - A larger stencil (i.e., solution based on more points, e.g., $j + 2, j + 1, j$) will create even more points on each side that we cannot solve



Boundary Conditions

- We specify what is happening at the extremes of our domains using boundary conditions
- These conditions should be based on the actual flow physics
- Even the best numerical schemes are absolutely useless if your boundary conditions do not properly reflect the actual system

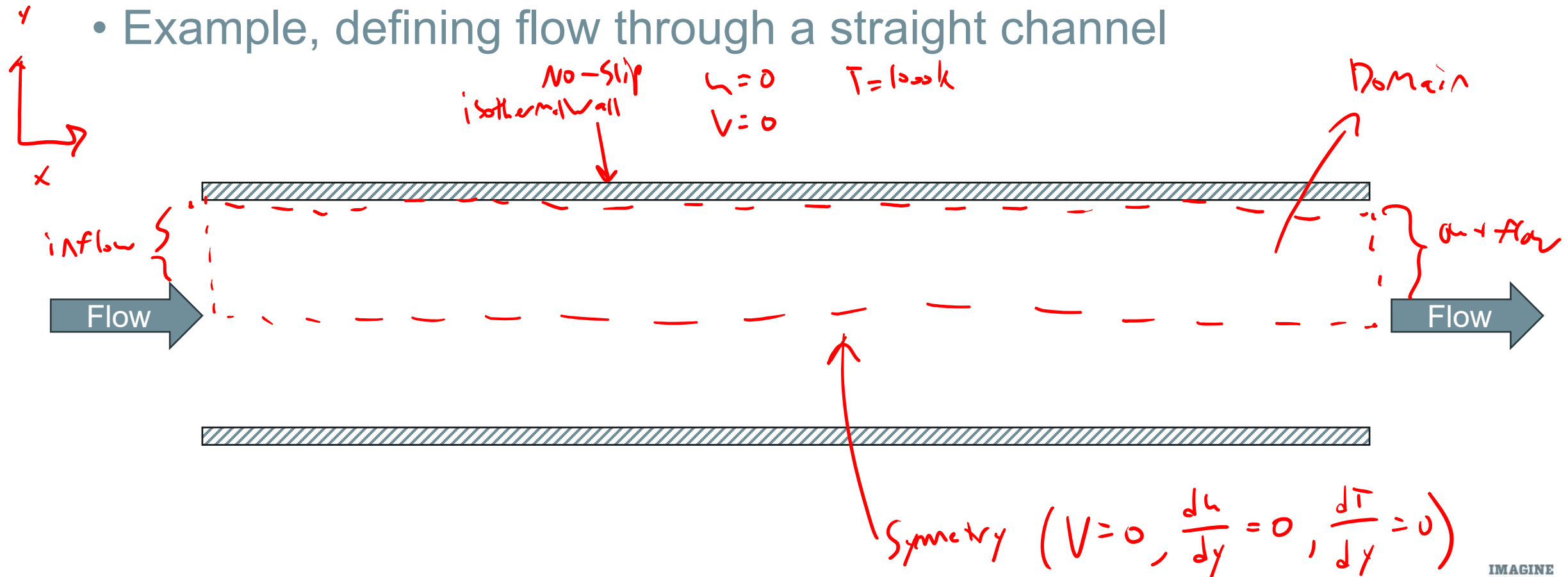
Boundary Conditions

- Physical boundary conditions we use in fluid mechanics:
 - Inlet/Inflow – Flow entering the domain across a boundary
 - Outlet/Outflow – Flow exiting the domain across a boundary
 - Wall – Solid boundary, no-slip condition
 - Sometimes we use slip boundaries if we're ignoring viscosity (e.g., Euler Eqns.)
 - Compressible flows need thermal BC at the wall. Usually adiabatic or isothermal
 - Symmetry
 - Periodic – Flow out of one boundary = flow into another
 - Far-field – Flow “unaffected” by CFD domain (i.e., far from any activity)
 - And many more
- Correct boundary conditions are extremely problem dependent

Boundary Conditions

Laminar (symmetric)
Boundary Layer thick compared to height
Wall is hotter than flow
Very short channel (not fully developed)

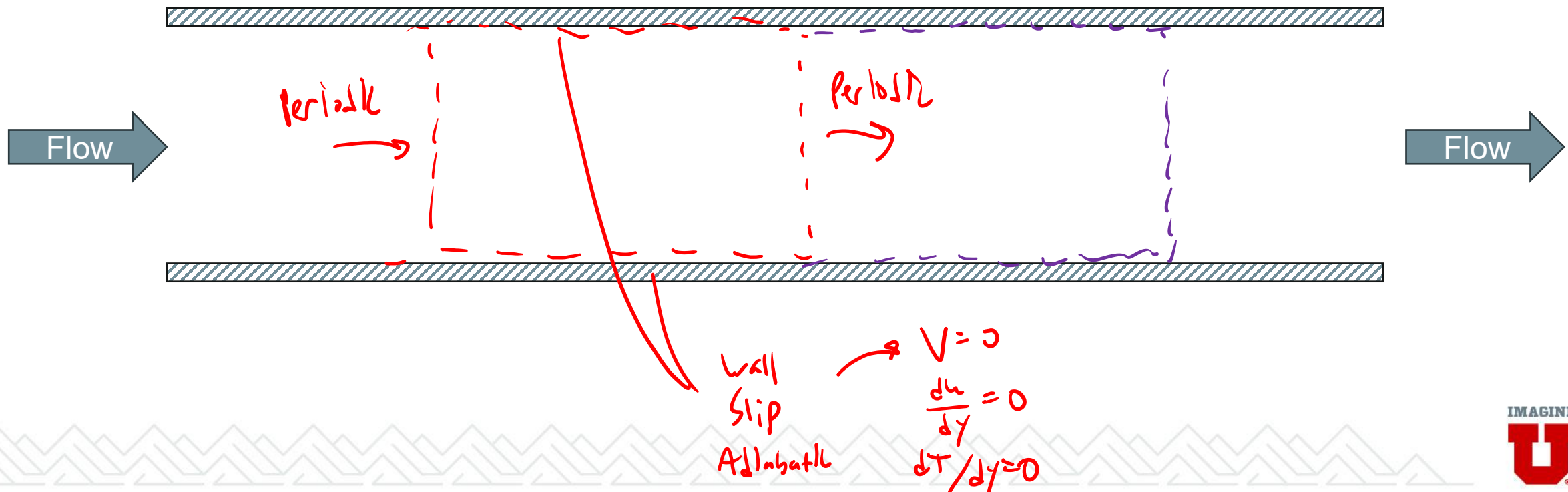
- Example, defining flow through a straight channel



Boundary Conditions

Turbulent (non-symmetric)
Boundary Layer very thin compared to height
Wall is similar temperature to flow
Very long channel (fully developed)

- Example, defining flow through a straight channel



Boundary Conditions

- How do these physical boundary conditions translate into CFD?
- Three primary types of mathematical boundary conditions we will work with (although more exist)
 - Dirichlet – prescribed value
 - Neumann – prescribed gradient
 - Periodic – matching two boundaries
- By using and combining these, we can recreate many of the physical boundary conditions we are interested in

Boundary Conditions

- Dirichlet boundary conditions specify the value of a variable

$$\phi = \phi_b$$

- Useful for setting a specific value of a variable at a boundary

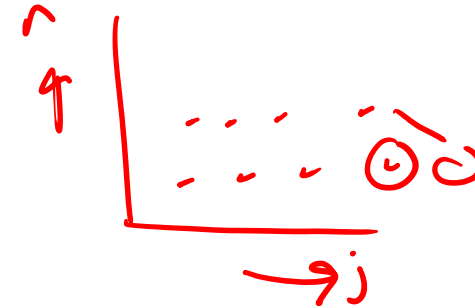
- No slip wall: $u_b = v_b = w_b = 0$
- Isothermal wall: $T_b = 300 \text{ K}$
- Inflow: $\rho_b = A, u_b = B, T_b = C$

- Note that these can vary in time/space

$$\phi(x, y, z, t) = \phi_b(x, y, z, t)$$

- For example, specify a parabolic inlet velocity profile; $u_b = f(y)$

Boundary Conditions



- Neumann boundary conditions specify the gradient of a variable

discretize $\rightarrow \frac{\partial \phi}{\partial n} = g$ For example $\frac{u_{j+1} - u_j}{\Delta n} = g$

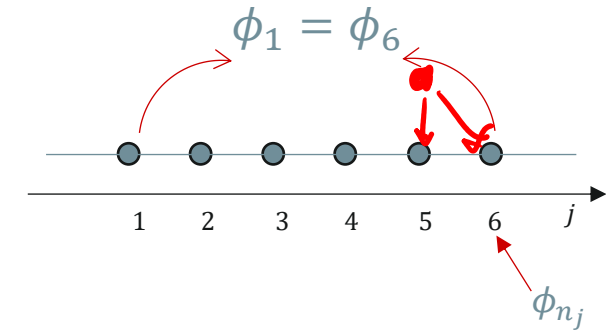
- Some examples of useful Neumann BCs:

- Adiabatic wall $\left(\frac{\partial T}{\partial n} = 0\right)$ or wall with prescribed heat flux $\left(\frac{\partial T}{\partial n} = g\right)$
- Symmetry is Dirichlet $(u_n = 0)$ and Neumann together $\left(\frac{\partial u_t}{\partial n} = 0\right)$
- Often used for outlets if you don't know the exact outlet behavior

$\downarrow$
 $u_{j+1} = u_j + g \Delta n$
 $j = n_{\text{points}}$

- Value can change as function of space and time

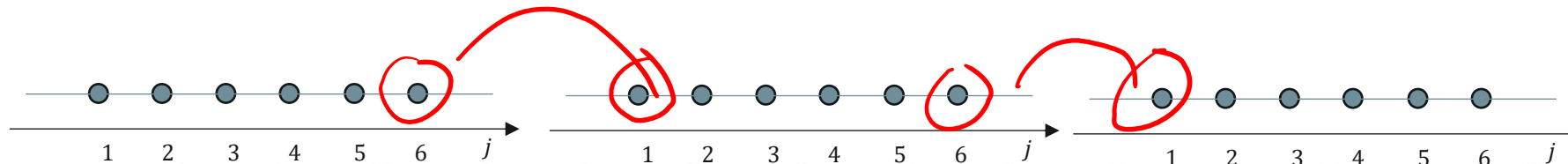
Boundary Conditions



- Periodic boundary conditions connect the values at one boundary to another

$$\phi_1 = \phi_{n_j} \text{ — \# points in } j$$

- Very common for fundamental studies, less so for applied
 - Basic code demonstrator (e.g., every video I've shown so far)
 - Homogeneous isotropic turbulence in a box
 - Quasi-steady problems (e.g., fully developed pipe flow)
 - Super common in “spectral” codes



Summary

- We've now covered all the fundamentals for anyone in this course to write their own “CFD” code – Assignment #1
 - What equations need to be solved?
 - How do we approximate those equations algebraically?
 - What kind of error do we expect from those approximations?
 - How do we prevent these approximations from being unstable
 - How, and how much will dissipation/dispersion modify the solutions?